Linux Commands and Windows Commands

Aim:

To study and execute various Linux commands For File and directory management, text processing and basic networking operation

→ Study of various Commands used in Linux pwd

Description: prints the current working directory

output:

```
$ pwd
/home/user
```

Description: Create a new Folder

output

```
$ mkdir Foldername
$ ls
Desktop Document Downloads ~~Fedo~~ Foldername
```

Description: Changes the current directory

output:

```
$ cd Foldname
```

Description: Creates an empty file

output:

```
$ touch example.txt
$ ls
```

Description: writes text into a file

Output:

```
$echo "Hi" > example.txt
```

Description: Displays the content of a file

output:

```
$ cat example.txt
Hi
```

Description: Searches for a specific pattern in a file

```
$ grep "Hi" example.txt
Hi
```

Description: Duplays the number of lines, words and byte in a file

Output

```
$ wc example.txt
1 1 3 example.txt
```

Description: Counts the number of lines in a File

Output
```
$ wc -l example.txt
1 example.txt
```

Description: Counts the number of words in a file

Output
```
$wc -w example.txt
1 example.txt
```

Description: Counts the number of bytes in a File

Output
```
$wc -c example.txt
3 example.txt
```

Description: Count the number of characters in a File

Output:
```
$ wc -m example.txt
3 example.txt
```

Description: Displays the length of the longest line in the file

Output
```
$ wc -L example.txt
2 example.txt
```

Basic Networking Commands

Description: Display the system's hostname

output:

```
$hostname
localhost - live
```

Description: Display the routing table

Output:

```
$route
kernel IP routing table
```

Description: Shows network connections and routing information

output:

```
$ netstat
Active internet connections
```

Description: Display the IP address and network interface details

output:

```
$ ip address show
1: lo
2: eth0
```

Description: Finds the IP address of a domain

output:

$ns lookup www.google.com

name: www.google.com

Address: 142.250.xxx.xxx

Description: Displays network interface configuration

output

$ ifconfig

en pos 3

inet 10.0.2.15

plng

Description: Tests connectivity to a host

output:

$ping www.google.com

Description: Display the ARP cache

Output

$ arp -a

gateway (10.0.2.2)

Result:

Thus the linux and windows commands are studied and executed sucessful.

Exp.no: 2

Date: 23/7/26

Learning and Assignment of IP Address manually to Computer

Aim:

To learn IP addressing and manually assign an IP address to a computer. To verify the assigned IP address using Linux networking Commands.

Theory:

An IP address is a unique numerical address assigned to every device connected to a network. It enables Communication between devices over a network. An IPV4 address consists of four part seperated by dots (eg. 192.168.1.10)

An Ip address can be assigned either manually (static IP) or automatically (Dynamic IP) using DHCP. A manually assigned IP address remains unless changed by the user. Along with the IP address, parameters such as the subnet Mask, Default Gateway and DNS Server are also Configured For proper network Communication

In Linux, the ip command is used to display and manage IP address, network interfaces, routing tables and neighbouring devices. It is one of the most commonly used commands for network configuration and troubleshooting.

Commands and Sample outputs

1. Display all IP address

Command:

ip address show

sample output

2: ens160:
inet 172.16.211.128/24
inet 6 fe80::20c:29ff::fe75:9927/64

observation:

Displays the IPV4 and IPV6 addresses of all networks

2) Display a specific network interface.

Command:

ip addr show dev ens160

Sample output:

2: ens160:
inet 172.16.211.128/24

3) Display Routing Table

Command

ip route show

Observation:

Displays the routing table and the default gateway.

4. Display Neigbar (ARP) table:

Command:

ip neigh show

Sample output:

172.16.211.1 Reachable
172.16.211.254 stable

Observation:

Displays the neighbouring device and their MAC address

Result:

The IP address was manually assigned and verified sucessfully using Linux networking Commands.

Exp.no: 3 Date: 23/7/26

Study of Different Types of Network Cables and Crimping of Cable with RJ45 Connector

Aim:
To study different types of networking cable and understand the crimping process using an RJ 45 connector.

Concept:
Network cables are used to Connect Computer and other networking devices For data Communication The most Commonly used cable in LAN is the unishielded Twisted Pair (UTP) cable. An RJ45 Connector is attached to the ends of the cable to establish a network Connection. The wires are arranged according to the T568A or T568B wiring standard before crimping and the Completed cable is tested to ensure proper Connectivity.

Types of Network Cables:

Cable Type	Description
1) UTP (unshielded Twisted Pair)	Commonly used for Ethernet LAN's and office networks
2) STP (shielded Twisted pair)	Used in area with high electrical or electromagnetic interface.
3) Coaxial Cable	Used for cable television, broadband internet and CCTV systems
4) Optical fiber cable	Used for high speed, high bandwidth and long distance communication

Crimping Procedure:

* Cut the UTP cable to the required length
* Strip about 2-3 cm of the outer insulation
* Untwist and straighten the wires
* Arrange the wires according to the T568B standard.
* Trim the wire end evenly
* Insert the wires into the RJ45 connector
* Crimp the connector using a crimping

* Test the cable using a LAN cable tester

Observation:

* The wires were arranged according to the 568B standard
* The RJ45 connector was clamped successfully
* The cable tester indicator ~~pe~~ proper continuity
* The cable was ready for network communication

Result:

The different types of network cables were studied and the UTP cable was successfully terminated using RJ45 connector and tested for proper connectivity.

Exp: 4 Implement Packet Sniffing in Wireshark

Aim:

To capture and analyse various network protocal packets like (TCP, UDP, ARP, DNS, HTTP, ICMP, DHCP) using wireshark and apply display Filters to inspect them.

Software Used:

* Wireshark Network Protocol Analyser

Procedure

General Steps (For all captures):

* Open Wireshark.
* Selet local Area Connection
* Go to Capture → options
* Set: Stop capture automatically after 100 packet.
* Click start to begin capture
* Use Filter bar to type specific protocols like trp, arp, etc
* Use statistics - Flow Graph (For TCP/DNS) to view packet Flow.
* Click File → Save to store the captured packets

Tasks & Outputs

1. Filter: Display TCP/UDP packet and Flow Graph.
   * Filter used: tcp or udp
   * Output: Displayed only TCP/UDP packets
   * Flow graph: Viewable from statistics → Flow Graph

2. Filter: Display only ARP packets
   * Filter used: arp
   * Output: Displayed only Address Resolution Protocol packet

3. Filter: Display only DNS packets and Flow Graph
   * Filter used: dns
   * Output: Displayed only DNS queries and responses

4. Filter: Display only HTTP packets
   * Filter used: http
   * Output: Captured HTTP Traffic (browser or server communication)

5. Filter: Display IP/ICMP packets
   * Filter used: icmp or ip
   * Output: Shows ICMP packets (used in ping operations) and IP packet info

6. Filter: Display only DHCP packets
   * Filter used: bootp (used for DHCP)
   * Output: Displays DHCP Discover, Offer, Request, Acknowledgement packet.

Result:

Sucessfully captured and filtered different protocol packets using wireshark and understand how to inspect and analyse them with tools like filter and flow graphs.

Exp: 5
24/7/26

Develop a customized ping Command to test the server connectivity

~~Why~~

Aim:

To develop a customized ping command to test the server connectivity

Why is Ping used?

Ping is a network diagnostic command used to check whether a server or host is reachable over a network. It helps verify connectivity between two devices and identify basic network issue.

What does ping measure?

* Reachability - Check whether destination server can be contacted
* Latency (Response Time) - Measures the time taken for packets to travel to the server and back (in milliseconds)
* Packet Loss - Shows whether any packets were lost during transmission.

Uses:

* Verify that a server is online
* Test network connectivity before troubleshooting application
* Measure network response time

* Detect packet loss caused by unstable connection

* Perform basic network health checks

Commands

1. Basic Ping Command

ping www.google.com

Output:

Displays the IP address of the server, the number of packets sent, received, lost and average round trip time

2. Ping Localhost (To check internal network stack):

ping localhost

Output:

Replies from 127.0.0.1 confirm that the TCP/IP stack ~~sup~~ on the local system is working properly.

3. Ping using IP address

ping 8.8.8.8

Output:

Displays connectivity and delay time (in milliseconds) between your system and Google DNS server

5. ping with Specific Number of Echo Requests

ping www.google.com -n 5

output:

Sends exactly 5 echo requests and displays the result with average time and packet loss summary.

6. Ping with packet Size Specification

ping www.google.com -l 1000

Output:

Sends a ping request with a packet size of 1000 bytes, showing transmission performance

Result:

The ping commands were sucessfully executed in windows Command Prompt. The network connectivity, latency and packet transmission status between the local computer and various servers were verified.

Exp: 7 Develop a Simple Calculator Using XML RPC

Aim:

To develop a simple calculator application using the XML-RPC protocol in python that performs basic arithmetic operation like addition, subtraction, Multiplication and division through client-server communication

Description:

XML-RPC is a protocol that uses XML to encode request and responses and HTTP as a transport Mechanism. It enables communication between a client and a server where the client invokes methods hosted on the server remotely

Code:

server.py

```
From xmlrpc.server import simpleXMLRPCServer

class Calculator
    def add(self, x, y):
        return x+y;

    def sub(self, x, y):
        return x-y

    def multi(self, x, y)
        return x*y
```

```
    def divide(self, x, y)
    if y!=0:
        return x/y

    else:
        return "Error: Division by zero is not
                allowed"

server = SimpleXMLRPCServer(("localhost", 8000))
server.register_instance(calculator())
server.serve_forever()
```

client.py

```
import xmlrpc.client
proxy = xmlrpc.client.ServerProxy("http://localhost:8000/")
print("Addition: 10+5 =", proxy.add(10,5)))
print("Subtraction: 10-5 =", proxy.sub(10,5))
print("Multiplication: 10*5 = " proxy.multiply(10,5))
print("Division: 10/5 =" proxy.divide(10,5))
```

Software Requirements

* OS: Windows/Linux
* Python version: 3.x or above
* Modules Required: xmlrpc.server.client
* Text Editor: Vs code.

Procedure:

* Open a Python IDE or terminal
* Create two seperate files server.py and client.py
* Write the provided server side code in server.py and client side code in client.py
* Run server.py file First
* Run client.py file to program arithmatic operations remotely
* Observe the results displayed on the client screen and verify the server output.

Result:

The XML RPC based calculator was sucessfully implemented. The client communicated with the server using XML-RPC, performed all arithmatic opertions ~~ato~~ remotely and displayed accurate results.

Exp: 9 Design a Simple Topology And Configure with one Router, Two Switches and PCs Using CISCO Packet Tracer

Aim:

To design and configure a simple network topology consisting of one router, two switches, and multiple PCs in Cisco packet Tracer, and enable communication between PCs

Definitions

*Routers: A networking device that forwards data packets between computer networks, directing trafic on the internet.

*Switch: A device that connects devices within a network and uses a MAC addresses to forward data to the correct destination

*PC (Personal Computer): End devices in a network that users interact with to send and received data.

*Cables:

i) Straight-through Cable: Used to connect different devices like PC to switch or switch to Router.

ii) Crossover Cable: Used to connect similar devices like switch to switch (rarely needed with modern switches)

iii) Console Cable: Used to connect PC to router or switch console port for Configuration

Equipment Needed

* Cisco packet Tracer Software
* 1 Router (eg: 2811)
* 2 Switches (eg: 2960)
* 4 PCs
* Appropriate cables (Copper Straight-Through)

Step-By-Step Procedure:

Step 1: Open Cisco Packet Tracer

* Launch Cisco packet tracer on your Computer

Step 2: Add devices to Workshop:

* From the bottom device toolbar, drag and drop:
  * 1 Router (1941)
  * 2 ~~sto~~ switches (2960)
  * 4 PCs (2 connected to each switch)

Step 3: Connect Devices with Cables

Use Copper Straight-Through cables to connect:

* Router Gigabit Ethernet 0/0 to switch0 FastEthernet 0/1
* Router Gigabit Ethernet 0/1 to switch1 Fast Ethernet 0/1

PCs to the switches

* PC0 to switch0 Fast Ethernet 0/2
* PC1 to switch0 Fast Ethernet 0/3
* PC2 to switch1 Fast Ethernet 0/2
* PC3 to switch1 Fast Ethernet 0/3

Step: 4 Configure Router Interface

*Click the Router

* Cli tab

*Enter the IP address and give no shutdown Command

Step 5: Configure PC IP addresses

*Click on Each PC

*Desktop tab

* IP configuration and assign IP addresses and gateways:

For Eg:

PC0 : 192.168.1.10 255.255.255.0

step 6: Save and Test Connectivity

*Save the Packet Tracer File

*Test the connectivity using the ping command from PC0 to PC2 or PC3

*Go to CMD type ping: 192.168.2.10

*You should see replies if the network is connected sucessfully.

Result:

The PCs connected to different switches can communicate with each other via the router. The sucessful ping replies verify that the network is properly Configured and operational.

Exp:10 Customize Switches with Network Modules Using Cisco Packet Tracer

Aim:

To learn how to add a network module to cisco switch, connect multiple PCs and a server, assign IP addresses and test connectivity in Cisco Packet Tracer.

Description:

This experiment is about building a simple computer network using a switch with extra module ports. You will connect five PCs and one server, assign IPs, and check if all devices can communicate. This helps you understand how modular switches are used to create flexible networks.

Definition:

A network module is an extra component you can add to a switch. It gives you more ports or add new types of connections. In packet Tracer, modules such as HWIC-4ESW let you expand the switch capabilities

Procedure:

1. Open Cisco Packet Tracer and set up your workspace
2. Drag and drop a 2960-24TT switch to the centre
3. Add five PCs and One Server-PT from the End Devices Panel.
4. Power Off the switch:
   - Click the Switch, open physical tab, turn off - power
5. Install HWIC-4ESW module to the switch
   - In physical tab, drag HWIC-4ESW into an empty module slot.
6. Power On the switch and let it boot
7. Connect all devices using Copper straight-Through cables:
   PC0 → Switch FastEthernet 0/1
8. Assign IP addresses to each devices
9. Test the connectivity (From any PC open CMD and ping other devices)

Output:

A working network setup where every PC and the server can sucessfully ping each other, proving that both build-in and module ports on the switch are functioning

Result:

The experiment demonstrates sucessful customization of a switch with a network module and connectivity between all devices. The network works as expected and all devices communicate.

EXPERIMENT 6

CS19541 – COMPUTER NETWORKS LAB

Error Detection and Correction Using Hamming Code

Aim

To implement error detection and correction using Hamming Code by converting text data into binary, adding redundant bits, transmitting the encoded data through a channel file, and correcting single-bit errors at the receiver.

Theory

Hamming (12,8) code uses 8 data bits and 4 parity bits. The parity bits are placed at positions 1, 2, 4 and 8. It can detect and correct a single-bit error in a 12-bit codeword.

Input

```
input.txt
HELLO

H = 01001000
E = 01000101
L = 01001100
L = 01001100
O = 01001111
```

Program 1: Sender

```
#include <stdio.h>

void encodeHamming(unsigned char data, char *code)
{
    int b[13] = {0};
    int i;

    for (i = 0; i < 8; i++)
        b[12 - i] = (data >> i) & 1;

    b[1] = b[3] ^ b[5] ^ b[7] ^ b[9] ^ b[11];
    b[2] = b[3] ^ b[6] ^ b[7] ^ b[10] ^ b[11];
    b[4] = b[5] ^ b[6] ^ b[7] ^ b[12];
    b[8] = b[9] ^ b[10] ^ b[11] ^ b[12];

    for (i = 1; i <= 12; i++)
        code[i - 1] = b[i] + '0';

    code[12] = '\0';
}

int main()
{
    FILE *input, *channel;
    unsigned char ch;
    char code[13];

    input = fopen("input.txt", "rb");
    if (input == NULL) {
        printf("Error: Cannot open input.txt\n");
        return 1;
    }

    channel = fopen("channel.txt", "w");
    if (channel == NULL) {
        printf("Error: Cannot create channel.txt\n");
        fclose(input);
        return 1;
    }

    while (fread(&ch, 1, 1, input) == 1) {
        encodeHamming(ch, code);
        fprintf(channel, "%s", code);
```

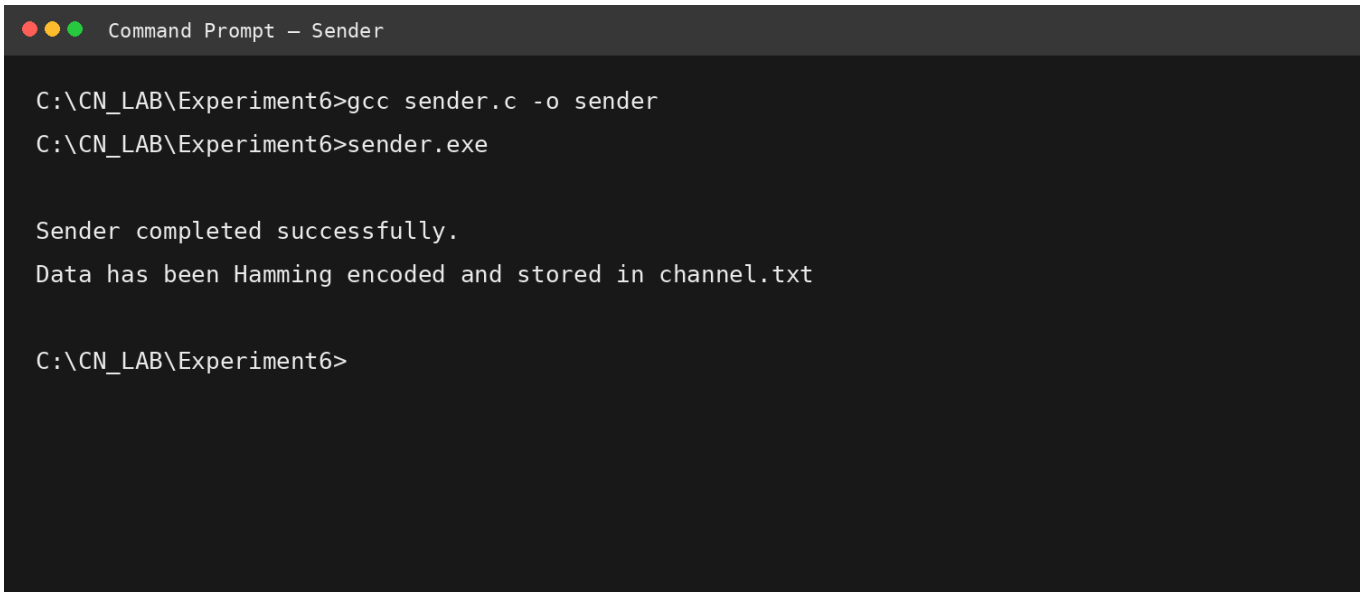
```
        }

        fclose(input);
        fclose(channel);

        printf("Sender completed successfully.\n");
        printf("Data has been Hamming encoded and stored in channel.txt\n");

        return 0;
    }
```

Sender – Compiled and Executed Output



Program 2: Receiver

```
#include <stdio.h>

int decodeHamming(char *code, unsigned char *data)
{
    int b[13] = {0};
    int i, syndrome = 0;

    for (i = 1; i <= 12; i++)
        b[i] = code[i - 1] - '0';

    if ((b[1]^b[3]^b[5]^b[7]^b[9]^b[11]) != 0)
        syndrome += 1;

    if ((b[2]^b[3]^b[6]^b[7]^b[10]^b[11]) != 0)
        syndrome += 2;

    if ((b[4]^b[5]^b[6]^b[7]^b[12]) != 0)
        syndrome += 4;

    if ((b[8]^b[9]^b[10]^b[11]^b[12]) != 0)
        syndrome += 8;

    if (syndrome != 0) {
        printf("Error detected at position %d\n", syndrome);
        if (syndrome <= 12) {
            b[syndrome] ^= 1;
            printf("Error corrected at position %d\n", syndrome);
        }
    }

    *data = 0;
    *data |= b[3] << 7;
    *data |= b[5] << 6;
    *data |= b[6] << 5;
    *data |= b[7] << 4;
    *data |= b[9] << 3;
    *data |= b[10] << 2;
    *data |= b[11] << 1;
    *data |= b[12];

    return syndrome;
}

int main()
{
    FILE *channel;
    char code[13];
    int ch, count = 0;
    unsigned char data;

    channel = fopen("channel.txt", "r");
    if (channel == NULL) {
        printf("Error: Cannot open channel.txt\n");
        return 1;
    }

    printf("Receiver started...\n\n");
    printf("Decoded Output:\n");

    while ((ch = fgetc(channel)) != EOF) {
        if (ch == '0' || ch == '1') {
            code[count++] = ch;

            if (count == 12) {
                code[12] = '\0';
                decodeHamming(code, &data);
                printf("%c", data);
                count = 0;
            }
        }
    }

    fclose(channel);

    printf("\n\nReceiver completed successfully.\n");
    return 0;
}
```

Receiver – Compiled and Executed Output

```
Command Prompt – Receiver

C:\CN_LAB\Experiment6>gcc receiver.c -o receiver
C:\CN_LAB\Experiment6>receiver.exe

Receiver started...

Decoded Output:
HELLO

Receiver completed successfully.
C:\CN_LAB\Experiment6>
```

Single-Bit Error Detection and Correction Output

After changing one bit in the transmitted Hamming code, the receiver detects the error position, corrects it, and recovers the original message.

```
Command Prompt – Hamming Error Correction

C:\CN_LAB\Experiment6>receiver.exe

Receiver started...

Decoded Output:
Error detected at position 5
Error corrected at position 5
HELLO

Receiver completed successfully.
```

Result

Thus, the Hamming (12,8) code was successfully implemented. The sender encoded the input text and stored it in channel.txt. The receiver successfully detected and corrected a single-bit error and recovered the original message HELLO.

Experiment 8 – Discover Live Hosts Using Nmap Scans

1. Aim

To discover live hosts using Nmap scans such as ARP, ICMP, TCP and UDP on the TryHackMe platform.

2. Platform Used

- TryHackMe
- AttackBox
- Nmap
- Wireshark

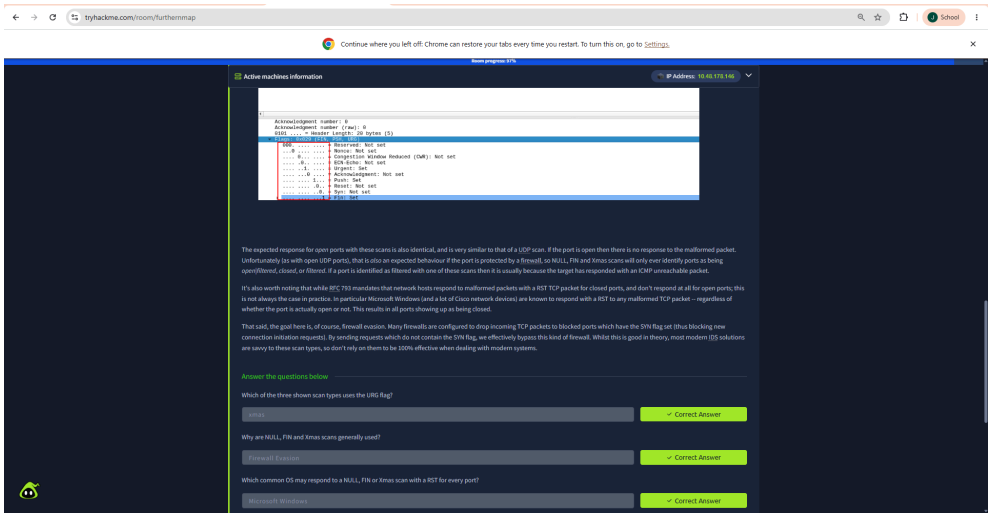
3. Practical Tasks Performed

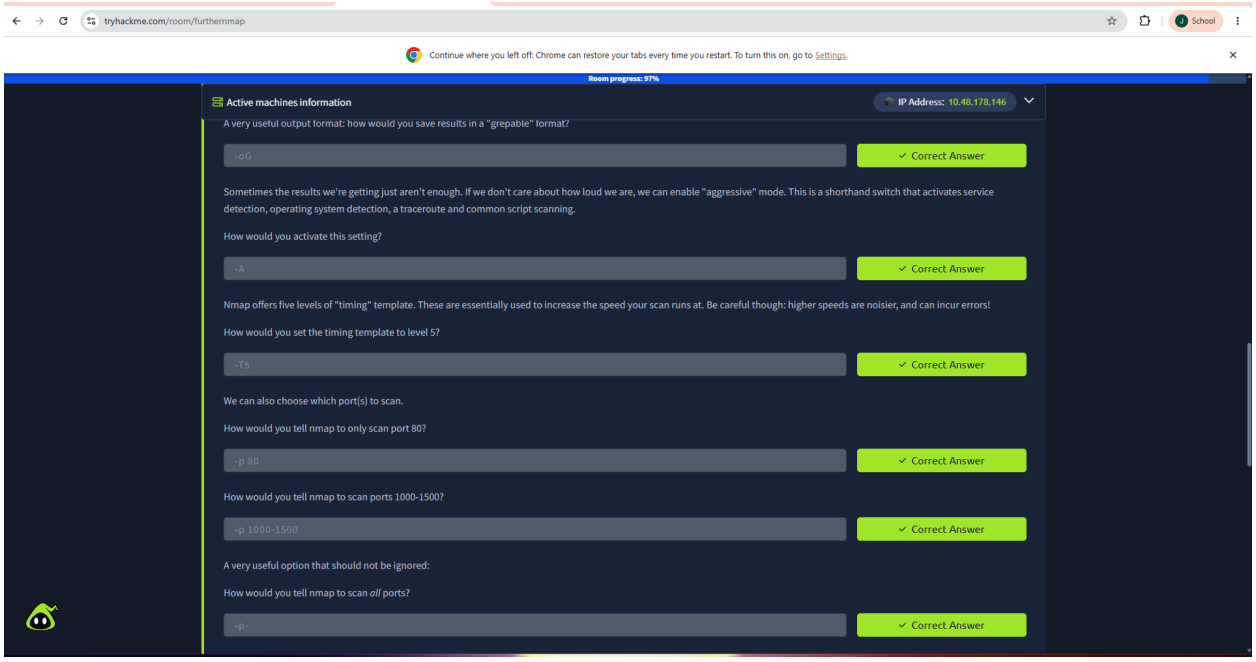
Task 2 – Introduction

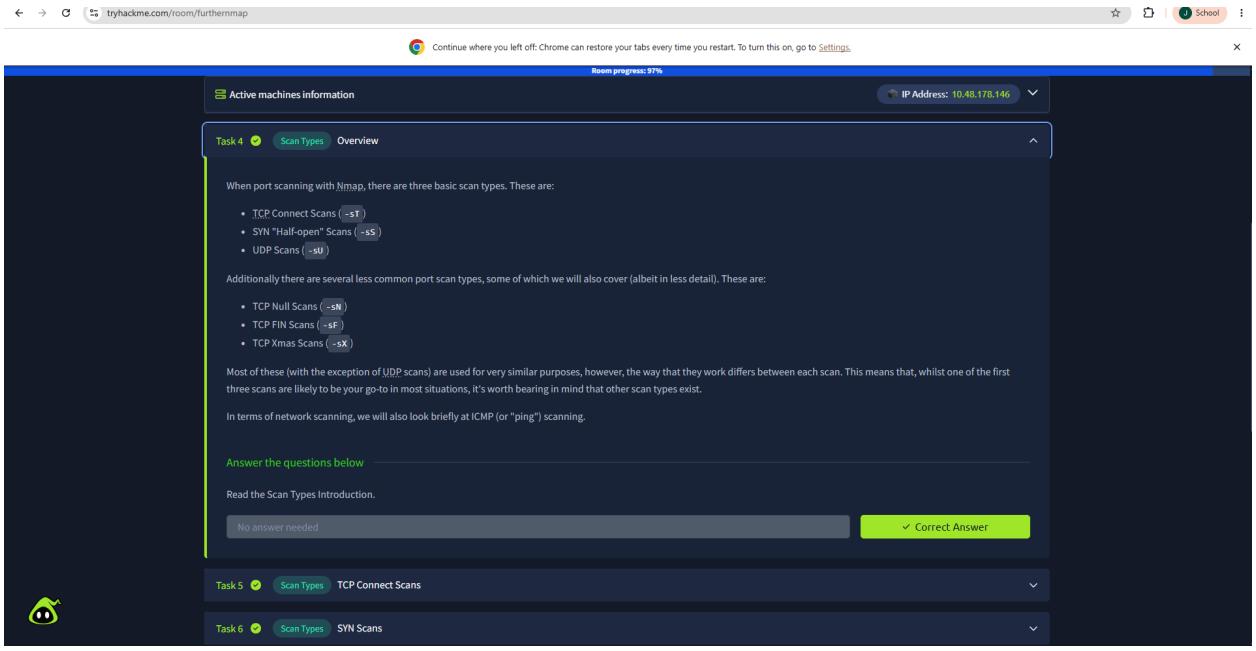
We learned:

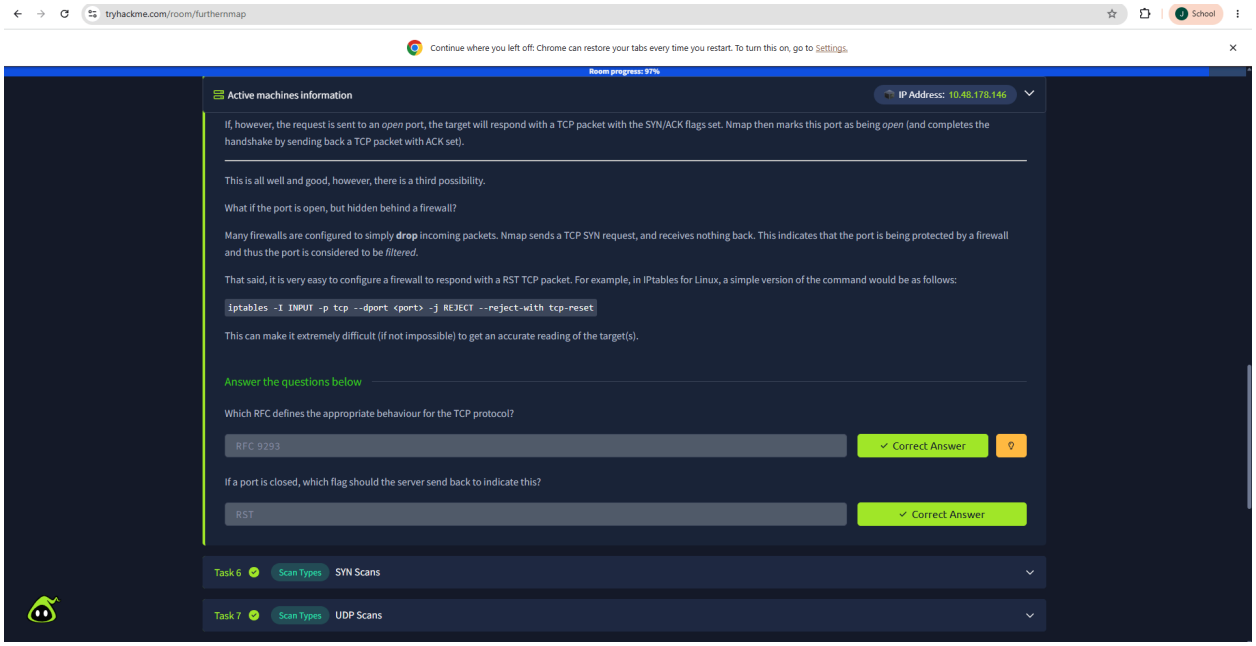
- Network ports
- Total number of ports: 65,535
- Well-known ports: 0–1023

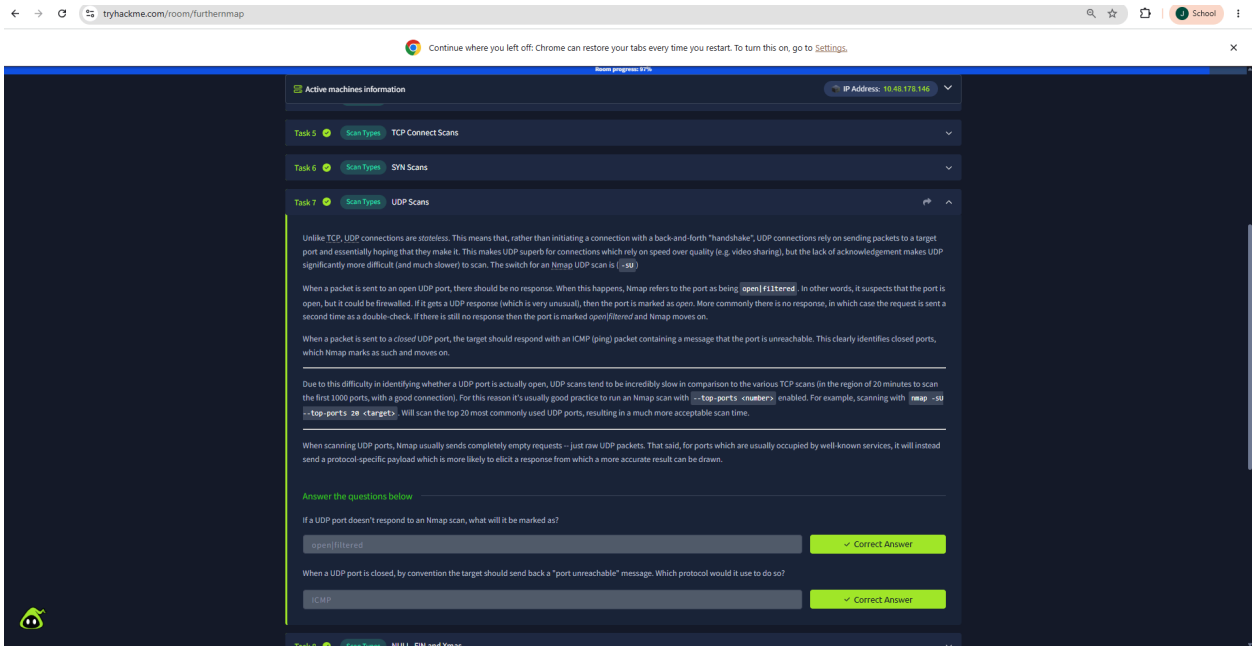
4. Screenshots

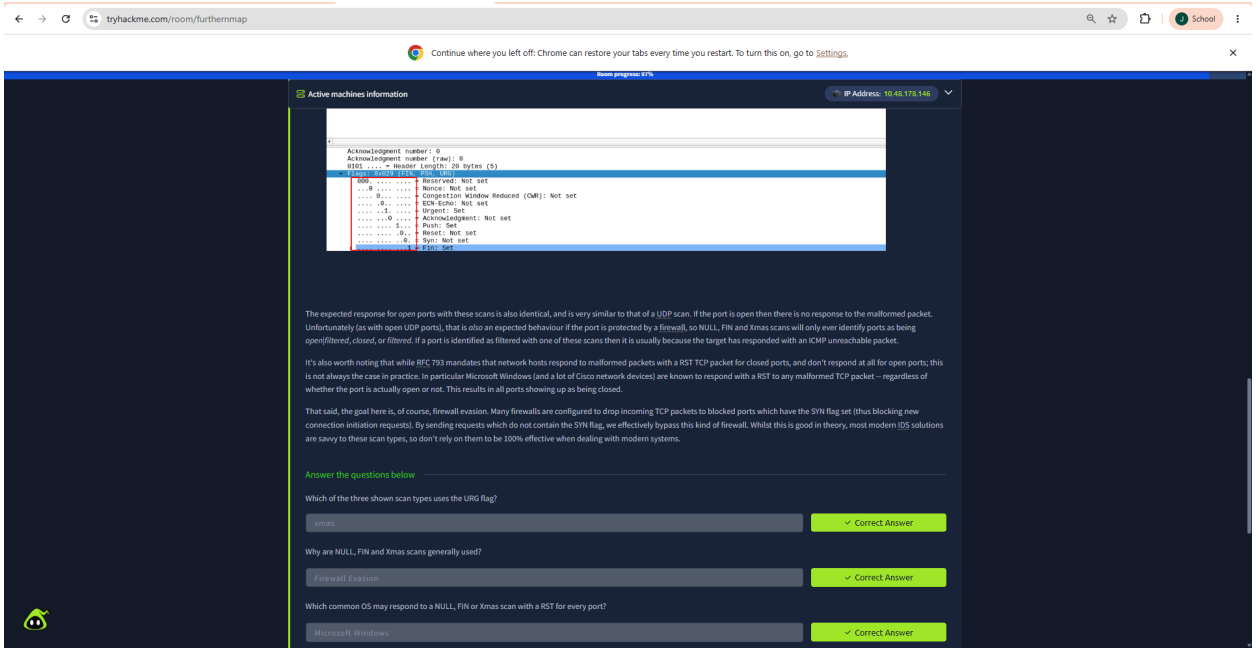



tryhackme.com/room/furthernmap
School
Continue where you left off: Chrome can restore your tabs every time you restart. To turn this on, go to Settings.
Room progress: 97%
Active machines information
IP Address: 10.48.178.146
A very useful output format: how would you save results in a "grepable" format?
-oG
Correct Answer
Sometimes the results we're getting just aren't enough. If we don't care about how loud we are, we can enable "aggressive" mode. This is a shorthand switch that activates service detection, operating system detection, a traceroute and common script scanning.
How would you activate this setting?
-A
Correct Answer
Nmap offers five levels of "timing" template. These are essentially used to increase the speed your scan runs at. Be careful though: higher speeds are noisier, and can incur errors!
How would you set the timing template to level 5?
-T5
Correct Answer
We can also choose which port(s) to scan.
How would you tell nmap to only scan port 80?
-p 80
Correct Answer
How would you tell nmap to scan ports 1000-1500?
-p 1000-1500
Correct Answer
A very useful option that should not be ignored:
How would you tell nmap to scan all ports?
-p-
Correct Answer


tryhackme.com/room/furthernmap
School
Continue where you left off: Chrome can restore your tabs every time you restart. To turn this on, go to Settings.
Room progress: 97%
Active machines information
IP Address: 10.48.178.146
Task 4
Scan Types
Overview
When port scanning with Nmap, there are three basic scan types. These are:
TCP Connect Scans (-sT)
SYN "Half-open" Scans (-sS)
UDP Scans (-sU)
Additionally there are several less common port scan types, some of which we will also cover (albeit in less detail). These are:
TCP Null Scans (-sN)
TCP FIN Scans (-sF)
TCP Xmas Scans (-sX)
Most of these (with the exception of UDP scans) are used for very similar purposes, however, the way that they work differs between each scan. This means that, whilst one of the first three scans are likely to be your go-to in most situations, it's worth bearing in mind that other scan types exist.
In terms of network scanning, we will also look briefly at ICMP (or "ping") scanning.
Answer the questions below
Read the Scan Types Introduction.
No answer needed
Correct Answer
Task 5
Scan Types
TCP Connect Scans
Task 6
Scan Types
SYN Scans


tryhackme.com/room/furthernmap
Continue where you left off: Chrome can restore your tabs every time you restart. To turn this on, go to Settings.
Room progress: 97%
Active machines information
IP Address: 10.48.178.146
If, however, the request is sent to an open port, the target will respond with a TCP packet with the SYN/ACK flags set. Nmap then marks this port as being open (and completes the handshake by sending back a TCP packet with ACK set).
This is all well and good, however, there is a third possibility.
What if the port is open, but hidden behind a firewall?
Many firewalls are configured to simply drop incoming packets. Nmap sends a TCP SYN request, and receives nothing back. This indicates that the port is being protected by a firewall and thus the port is considered to be filtered.
That said, it is very easy to configure a firewall to respond with a RST TCP packet. For example, in IPtables for Linux, a simple version of the command would be as follows:
iptables -I INPUT -p tcp --dport <port> -j REJECT --reject-with tcp-reset
This can make it extremely difficult (if not impossible) to get an accurate reading of the target(s).
Answer the questions below
Which RFC defines the appropriate behaviour for the TCP protocol?
RFC 9293
Correct Answer
If a port is closed, which flag should the server send back to indicate this?
RST
Correct Answer
Task 6 Scan Types SYN Scans
Task 7 Scan Types UDP Scans


tryhackme.com/room/furthernmap
Continue where you left off: Chrome can restore your tabs every time you restart. To turn this on, go to Settings.
Room progress: 97%
Active machines information
IP Address: 10.48.178.146
Task 5 Scan Types TCP Connect Scans
Task 6 Scan Types SYN Scans
Task 7 Scan Types UDP Scans
Unlike TCP, UDP connections are stateless. This means that, rather than initiating a connection with a back-and-forth "handshake", UDP connections rely on sending packets to a target port and essentially hoping that they make it. This makes UDP superb for connections which rely on speed over quality (e.g. video sharing), but the lack of acknowledgement makes UDP significantly more difficult (and much slower) to scan. The switch for an Nmap UDP scan is -sU
When a packet is sent to an open UDP port, there should be no response. When this happens, Nmap refers to the port as being open|filtered. In other words, it suspects that the port is open, but it could be firewalled. If it gets a UDP response (which is very unusual), then the port is marked as open. More commonly there is no response, in which case the request is sent a second time as a double-check. If there is still no response then the port is marked open|filtered and Nmap moves on.
When a packet is sent to a closed UDP port, the target should respond with an ICMP (ping) packet containing a message that the port is unreachable. This clearly identifies closed ports, which Nmap marks as such and moves on.
Due to this difficulty in identifying whether a UDP port is actually open, UDP scans tend to be incredibly slow in comparison to the various TCP scans (in the region of 20 minutes to scan the first 1000 ports, with a good connection). For this reason it's usually good practice to run an Nmap scan with --top-ports <number> enabled. For example, scanning with nmap -sU --top-ports 20 <target>. Will scan the top 20 most commonly used UDP ports, resulting in a much more acceptable scan time.
When scanning UDP ports, Nmap usually sends completely empty requests -- just raw UDP packets. That said, for ports which are usually occupied by well-known services, it will instead send a protocol-specific payload which is more likely to elicit a response from which a more accurate result can be drawn.
Answer the questions below
If a UDP port doesn't respond to an Nmap scan, what will it be marked as?
open|filtered
Correct Answer
When a UDP port is closed, by convention the target should send back a "port unreachable" message. Which protocol would it use to do so?
ICMP
Correct Answer



5. Result

Thus, the required Nmap scans and host-discovery activities were successfully performed on the TryHackMe platform.

CS19541 – COMPUTER NETWORKS LAB

Practical 11 – Static Routing and RIP

AIM

To configure and verify static routing and RIP routing between networks using Cisco Packet Tracer.

REQUIREMENTS

Cisco Packet Tracer, routers, PCs, and appropriate Ethernet/serial cables.

PROCEDURE

1. Place three routers and the required PCs in the workspace.
2. Connect the devices using suitable cables.
3. Assign IP addresses to router interfaces and PCs, including default gateways.
4. Enable interfaces using the **no shutdown** command. Configure clock rate on DCE serial interfaces when required.
5. Configure static routes using **ip route** and, where required, primary and backup paths.
6. Verify static routes using **show ip route static**.
7. Enable RIP using **router rip** and advertise directly connected networks using **network** commands.
8. Test connectivity using ping and tracert.

SAMPLE STATIC ROUTE

```
ip route [destination-network] [subnet-mask] [next-hop-IP]
```

SAMPLE RIP CONFIGURATION

```
router rip
version 2
network [network-address]
no auto-summary
```

INPUT

Network addresses, subnet masks, next-hop addresses, and router interface configurations.

OUTPUT

Routing tables display the configured static routes or RIP-learned routes. Ping and tracert confirm the path and connectivity between PCs.

OBSERVATION

1. Three routers and the required PCs were interconnected in Cisco Packet Tracer, and IP addresses were assigned to all interfaces and PCs.
2. Router interfaces were enabled using **no shutdown**, and DCE serial interfaces were configured with suitable clock rate and bandwidth values.
3. Static routes with primary and backup paths were configured using **ip route** and verified using **show ip route static**.
4. RIP was enabled using **router rip**, and directly connected networks were advertised using **network**.
5. Network connectivity was verified using ping and tracert, and communication between PC0 and PC1 was successfully established.

RESULT

Static routing and RIP were configured and tested successfully in Cisco Packet Tracer.

CS19541 – COMPUTER NETWORKS LAB

Practical 12 – Echo and Chat Client-Server Using TCP/UDP Sockets

AIM

To implement echo and chat client-server applications using TCP/UDP sockets.

PART A – ECHO CLIENT-SERVER

Algorithm:
1. Create a server socket and bind it to an address and port.
2. Listen for and accept a client connection when using TCP.
3. The client sends a message to the server.
4. The server receives the message and sends the same message back.
5. The client receives and displays the echoed message.

PART A – INPUT / OUTPUT

Input: A message entered by the client.
Output: The same message is returned by the server and displayed by the client.

PART B – CHAT CLIENT-SERVER

Algorithm:
1. Create and configure server and client sockets.
2. Establish a connection for TCP, or use sendto/recvfrom for UDP.
3. Allow the client and server to send and receive messages alternately.
4. Display received messages on each side.
5. Stop communication when either side enters “exit”.

PART B – INPUT / OUTPUT

Input: Messages entered by the client and server.
Output: Messages are exchanged and displayed at both ends until communication is terminated.

PROCEDURE

1. Save the client and server programs in separate Python files, or use a combined program that lets the user choose the role.
2. Run the server first.
3. Run the client in another terminal on the same machine or a reachable host.
4. Enter a message and observe the echo/chat response.
5. Enter “exit” to terminate the chat.

OBSERVATION

The echo server returned the message sent by the client. In the chat application, messages were exchanged between the client and server using socket communication.

RESULT

Echo and chat client-server applications were implemented and tested using socket programming.

CS19541 – COMPUTER NETWORKS LAB

Practical 13 – Implement Your Own Ping Program

AIM

To implement a simple ping program to check host reachability and measure round-trip time.

ALGORITHM

1. Read a host name or IP address from the user.
2. Resolve the host name to an IP address.
3. Create a raw socket using the ICMP protocol.
4. Construct and send an ICMP Echo Request packet.
5. Wait for an Echo Reply and measure the round-trip time.
6. Display the reply time or a timeout message, repeat as required, and close the socket.

PYTHON CODE

```
import socket
import struct
import time

host = input("Enter host name or IP address: ")
ip = socket.gethostbyname(host)
sock = socket.socket(socket.AF_INET, socket.SOCK_RAW, socket.IPPROTO_ICMP)
sock.settimeout(2)

for i in range(4):
packet_id = i
packet = struct.pack("!BBHHH", 8, 0, 0, packet_id, 1)
checksum = 0
for j in range(0, len(packet), 2):
checksum += (packet[j] << 8) + packet[j + 1]
checksum = (checksum >> 16) + (checksum & 0xffff)
checksum = ~checksum & 0xffff
packet = struct.pack("!BBHHH", 8, 0, checksum, packet_id, 1)
start = time.time()
sock.sendto(packet, (ip, 0))
try:
sock.recvfrom(1024)
end = time.time()
print("Reply from", ip, "time =", round((end - start) * 1000, 2), "ms")
except socket.timeout:
print("Request timed out")
sock.close()
```

INPUT

Enter a host name or IP address, for example: 8.8.8.8

OUTPUT

The program displays replies with round-trip time in milliseconds or “Request timed out” if no response is received.

NOTE

Raw ICMP sockets may require Administrator/root privileges. Some hosts or networks block ICMP packets, so a timeout does not always mean that the host is unavailable.

RESULT

A basic ping program was implemented to send ICMP requests and display response times.

CS19541 – COMPUTER NETWORKS LAB

Practical 14 – Packet Sniffing Using RAW Sockets

AIM

To write a program using RAW sockets to capture and display network packet information.

ALGORITHM

1. Import the socket module.
2. Create a raw socket using the appropriate address family and protocol.
3. Run the program with the required permissions.
4. Capture packets arriving at the system's network interface.
5. Display packet source information and packet length.
6. Stop capture and close the socket.

PYTHON CODE – WINDOWS

```
import socket

s = socket.socket(socket.AF_INET, socket.SOCK_RAW, socket.IPPROTO_IP)
s.bind(("0.0.0.0", 0))
s.setsockopt(socket.IPPROTO_IP, socket.IP_HDRINCL, 1)
s.ioctl(socket.SIO_RCVALL, socket.RCVALL_ON)
print("Packet sniffing started...")
try:
while True:
data, addr = s.recvfrom(65535)
print("Packet received from:", addr[0])
print("Packet length:", len(data))
print("-" * 40)
except KeyboardInterrupt:
print("\nSniffing stopped.")
finally:
s.ioctl(socket.SIO_RCVALL, socket.RCVALL_OFF)
s.close()
```

INPUT

Network packets visible to the selected system network interface.

OUTPUT

The program displays the source IP address and length of each captured packet.

NOTE

This example uses Windows-specific raw-socket behavior and generally must be run as Administrator. Capture only traffic you are authorized to inspect.

RESULT

A basic packet-sniffing program using raw sockets was implemented to capture and display packet information.

CS19541 – COMPUTER NETWORKS LAB

Practical 15 – Web Log Analysis Using Webalizer

AIM

To analyze different types of web logs using the Webalizer tool.

REQUIREMENTS

Webalizer Windows version, a web server log file, Windows operating system, and a web browser.

PROCEDURE

1. Run the Webalizer Windows version.
2. Download or obtain a sample web server log file.
3. Select the log file and configure the required settings.
4. Click **Run Webalizer** to process the log file.
5. Open the generated HTML report in a web browser and inspect the statistics.

WEB LOG

A web log is a file that records requests made to a web server, including client IP address, date and time, requested URL, HTTP status code, and amount of data transferred.

WEBALIZER

Webalizer is a web log analysis tool that converts web server log files into statistical and graphical HTML reports.

PARAMETERS ANALYZED

- Number of visitors and visits
- Number of requests and pages viewed
- Data transferred
- Frequently accessed URLs
- Referring websites
- HTTP status codes
- Daily, monthly, and hourly traffic statistics

INPUT

A web server log file containing request and traffic information.

OUTPUT

Webalizer generates an HTML report showing visitor counts, requests, pages, traffic volume, popular URLs, referrers, and usage statistics.

OBSERVATION

The Webalizer tool processed the given web log file and generated statistical reports showing website traffic, visitors, requests, pages accessed, and data transfer details.

RESULT

The web log file was successfully analyzed using Webalizer, and the corresponding statistical and graphical reports were generated.